LOOP

PRAKASH KUMAR
BCA (DS & AI)
ENROLL :- A354187026010

Q1. What is a loop? Why do we need it?

A loop is used to execute a block of code repeatedly until a given condition is met or a sequence is completed.

- To avoid writing the same code again and again.
- To reduce code length.
- To save time and effort.
- To perform repetitive tasks easily.

Example:

```
for i in range(5):
    print("Hello")
```

Output:

```
Hello
Hello
Hello
Hello
Hello
```

Q2. What is the difference between a for loop and a while loop?

Both loops are used to repeat code, but they work differently.

For Loop	While Loop
Used to iterate over a sequence or range.	Used to repeat code while a condition is True.
Commonly used when the number of iterations is known.	Commonly used when the number of iterations is unknown.
Automatically takes the next item.	Usually requires manual updating of the condition variable.

Example of for loop:

```
for i in range(1, 4):    print(i)
```

Example of while loop:

```
i = 1while i <= 3:    print(i)    i += 1
```

Output of both:

1

2

Q3. What does range() do? Explain start, stop and step. The range() function generates a sequence of numbers. It is commonly used with a for loop.

Syntax:

range(start, stop, step)

Parameter	Meaning
start	Starting number (default is 0).
stop	Ending limit (not included).

for i in range(2, 11, 2): print(i)

Output:

2

4

6

8

10

- 2 → Start counting from 2.
- 11 → Stop before 11.
- 2 → Increase by 2 each time.

The stop value is never included in the generated sequence.

Q4. What is the difference between break and continue?

Both statements control the execution of a loop, but they have different purposes.

break	continue
Stops the entire loop immediately.	Skips only the current iteration.
Control moves outside the loop.	Control moves to the next iteration.

Example 1: break

```
for i in range(1, 6):    if i == 3:        break    print(i)
```

Output:

1

2

The loop stops completely when i becomes 3.

Example 2: continue

```
for i in range(1, 6):    if i == 3:        continue    print(i)
```

Output:

1

2

4

5

Here, only number 3 is skipped. The loop continues with the remaining numbers.

Remember:

- break = Exit the loop.
- continue = Skip the current iteration.

Q5. What is the pass statement? How is it different from continue?

The pass statement is a placeholder in Python. It does nothing when executed. It is used when a statement is required but we do not want to write any code yet.

Difference between pass and continue:

pass	continue
Does nothing.	Skips the current iteration.
Execution continues normally.	Moves to the next iteration of the loop.
Used as a placeholder.	Used to skip specific iterations.

Example of pass:

```
for i in range(1, 5):    if i == 2:        pass    print(i)
```

Output:

1

2

3

4

Here, pass does not affect the loop. All numbers are printed.

Example of continue:

```
for i in range(1, 5):    if i == 2:        continue    print(i)
```

Output:

1

3

4

Q6. Can a loop have an else block? When does it run?

Yes, Python allows an else block with both for and while loops.

The else block executes when the loop finishes normally without encountering a break statement.

Example:

for i in range(1, 4): print(i)else: print("Loop completed")

Output:

1

2

3

Loop completed

Explanation: The loop completes all iterations without using break, so the else block executes.

Example with break:

for i in range(1, 5): if i == 3: break print(i)else: print("Loop completed")

Output:

1

2

Here, the else block does not execute because the loop stops using break.

Q7. What is a nested loop? How many times does the inner body run?

A nested loop is a loop written inside another loop.

- The outside loop is called the outer loop.
- The inside loop is called the inner loop.
- The inner loop completes all its iterations for every single iteration of the outer loop.

Example:

```
for i in range(1, 4):    for j in range(1, 3):        print(i, j)
```

Output:

```
1 1
1 2
2 1
2 2
3 1
3 2
```

Explanation:

- Outer loop runs 3 times.
- Inner loop runs 2 times for each outer iteration.
- Total executions of the inner body = $3 \times 2 = 6$ times.

Q8. In a nested loop, if you use break in the inner loop, what happens to the outer loop?

When break is used inside the inner loop, it stops only the inner loop. The outer loop continues with its next iteration.

Example:

```
for i in range(1, 4):    for j in range(1, 4):        if j == 2:            break        print(i, j)
```

Output:

1 1

2 1

3 1

Explanation:

1. The outer loop starts with i = 1.
2. The inner loop stops when j = 2.
3. The outer loop continues with i = 2.
4. The same process repeats until the outer loop finishes.

Q9. What is an infinite loop? How do you avoid it, and how do you stop one?

An infinite loop is a loop that keeps running because its stopping condition never becomes False.

It usually happens when the condition is always True or the loop variable is not updated correctly.

Example:

```
i = 1while i <= 5:    print(i)
```

Here, the value of i never changes, so the loop keeps running.

How to avoid an infinite loop:

- Write a proper stopping condition.
- Update the loop variable correctly.
- Use break when a specific condition is met.

Correct example:

```
i = 1while i <= 5:    print(i)    i += 1
```

Output:

1

2

3

4

5

Q10. How do you loop backwards, from 10 down to 1?

We can use the range() function with a negative step to loop backwards.

Example:

```
for i in range(10, 0, -1):    print(i)
```

Output:

10

9

8

7

6

5

4

3

2

1

Explanation:

- 10 → Starting number.
- 0 → Stopping limit (not included).
- -1 → Decreases the number by 1 each time.

Q11. How do you get both the position and the item while looping over a list?

We use the enumerate() function to get both the index (position) and the item (value) while looping through a list.

Syntax:

enumerate(iterable, start=0)

By default, indexing starts from 0.

Example:

```
fruits = ["Apple", "Banana", "Mango"]for index, fruit in enumerate(fruits):    print(index, fruit)
```

Output:

0 Apple

1 Banana

2 Mango

Explanation:

- index stores the position of each item.
- fruit stores the actual item from the list.
- enumerate() provides both values together.

Q12. How do you loop through a dictionary? In Python, we can use a for loop to iterate through a dictionary.

A dictionary stores data in key-value pairs.

There are three common ways to loop through a dictionary.

Example dictionary:

```
student = {    "name": "Rahul",    "age": 20,    "course": "BCA"}
```

1. Loop through keys using keys():

```
for key in student.keys():    print(key)
```

Output:

name

age

course

2. Loop through values using values():

```
for value in student.values():    print(value)
```

Output:

Rahul

20

BCA

3. Loop through both keys and values using items():

```
for key, value in student.items():    print(key, ":", value)
```

Output:

name : Rahul

age : 20

course : BCA